



War modding

War modding involves modifying how war is treated in Europa Universalis V - the creation of new casus belli and their wargoal definitions - as well as creation of new, scripted peace treaties to go with it.

Please help with verifying or updating older sections of this article.
At least some were last verified for version 1.0.

İçindekiler

Casus belli

- [Technical details](#)
- [Syntax](#)
- [Wargoal type](#)
- [Creating casus belli](#)
- [Expiration date](#)
- [Declaring the war](#)
- [War parameters](#)
- [Additional parameters](#)
- [AI](#)
- [Required casus belli types](#)
- [Localisation](#)
- [Icon](#)

Wargoal types

- [Technical details](#)
- [Syntax example](#)
- [Ticking warscore](#)
- [War name](#)
- [Peace parameters for attacker and defender](#)
 - [Syntax](#)
 - [Costs](#)
 - [Allowed locations and subjugation](#)

[Icon](#)**[Scripted peace treaties](#)**[Technical details](#)[Syntax](#)[Requirements and effects](#)[Peace treaty category](#)[AI](#)[Peace treaty warscore and antagonism cost](#)[Interaction targets in peace treaties](#)[Scopes available in scripted peace treaties](#)[Additional fields](#)[Localisation](#)[Icon](#)**[References](#)**

Casus belli

Casus belli are the primary way of going to war. Modding casus bellis involves changing parameters involved in the acquisition of the casus belli, but also include many aspects related to the war itself.

Technical details

Casus bellis are located in `common/casus_belli`, usually in the `in_game` top folder.

For example:

```
common/casus_belli/example_file.txt
```

Syntax

```
cb_dissolve_tatar_yoke = {
    visible = {
        international_organization:tatar_yoke ?= {
            country_has_special_status = {
                type = special_status:tatar_tax_collector
                country = root
            }
        }
        scope:target = c:GLH
    }
    allow_creation = {
        country_rank = country_rank:rank_kingdom
    }

    war_goal_type = dissolve_tatar_yoke_wargoal
}
```



Wargoal type

All casus belli types need to have a wargoal type assigned using `war_goal_type`.

Creating casus belli

If the created casus belli is intended to be acquired through the GUI for creating casus belli, it should be supplied with `visible` and `allow_creation` parameters that will return true when the casus belli can be created. Both of the triggers are supplied with `ROOT` representing the fabricating country and `scope:target` as the target country.

`speed = <floating point> number` is then used to define the speed of casus belli creation. A casus belli is created when the progress reaches 100 and there is no base speed, so it is required to set a speed so that the game does not have to fall back onto a very low minimum.

Expiration date

Once a casus belli is acquired, it can be set to never expire using `can_expire = no`. Otherwise, the duration of how long a casus belli lasts can be set using `years`, `months`, `weeks` or `days`, all scripted values where `ROOT` is the nation receiving the casus belli and `scope:target` is the target country.

If that too, is not provided, the game will then add the casus belli for the duration set by CASUS_BELLI_MONTHS define (Vanilla value: 120).

Declaring the war

The ability to declare war with this casus belli can be limited using `allow_declaration` trigger. This trigger is supplied with `ROOT`, representing the declaring country, and `scope:target`, the target country.

Moreover, some wargoal types will expect you to select a province. To define province selection logic, `province` trigger is used. This trigger is supplied with `ROOT`, representing the province, with `scope:actor` representing declaring country and `scope:target` representing the country getting declared on.

War parameters

Casus bellis, as well as their selected wargoal types, provide extra control for what is possible within a war.

`allow_separate_peace = no` can be used to disable separate peace deals, but will also disable taking treaties from other recipients in the main peace deal.

`max_war_score_from_battles` is a floating point value that is used to set the max attainable war score from battles. If not set, the game will use WARSCORE_MAX_FROM_BATTLES define (Vanilla value: 50) instead.



Attackers and defenders get their own variations which stack with the prior using `additional_war_enthusiasm_attacker` and `additional_war_enthusiasm_defender`.

`antagonism_reduction_per_warworth_defender` is a script value that determines the % antagonism reduction based on the location taken warworth. `ROOT` is the country giving the land, `scope:recipient` is the country taking the land, and `scope:war` represents the war this is evaluated in.

Additional parameters

`no_cb = yes` is used to designate this casus belli as the eponymous "No casus belli".

`trade = yes` is used to set this casus belli as trade related. Whether a casus belli is trade related can be checked via `is_trade_cb` trigger. Designating a cb as trade cb will disable many peace relating to taking land, subjugation and royal marriage.

`cut_down_in_size_cb = yes` will make AI choose release nation peace treaties more often.

`allow_release_areas = yes` will allow a peace offer to release areas as sovereign nations.

AI

The decision for which casus belli gets selected by the AI can be modified using `ai_selection_desire`, which is a script value where `ROOT` is the AI country planning to declare war. The target is NOT accessible here. Typically, strong casus bellis are given a high value here.

`ai_subjugation_desire` can be used to define AI reasons for subjugating certain countries when using this casus belli. The field is a script value with the following provided:

- `ROOT` is the country subjugating.
- `scope:recipient` is subjugated country.
- `scope:subject_type` is evaluated subject type.
- `scope:war` is the war in which this evaluation is taking place.

`ai_cede_location_desire` can be used to define AI reasons for conquering certain locations when using this casus belli. The field is a script value with the following provided:

- `ROOT` is the country conquering.
- `scope:location` is the considered location.
- `scope:war` is the war in which this evaluation is taking place.

`allow_ports_for_reach_ai = no` can be used to make AI more self-conscious about taking land overseas it would have little control over.

Required casus belli types

Some casus bellis are used internally in the game but are open for scripting. Those casus bellis should not be removed, otherwise the game will error and might behave unexpectedly:

`cb_none`



- `cb_declined_call_to_arms`
- `cb_declined_call_to_arms`
- `cb_war_from_event`
- `cb_conquer_province`
- `cb_crusade`
- `cb_deus_vult`
- `cb_disrupting_trade`
- `cb_destroyed_building`
- `cb_exploration`
- `cb_independence_war`
- `cb_coalition`
- `cb_following_through_on_threat`
- `cb_flower_wars`
- `cb_claim_throne`
- `cb_resist_annexation`

Localisation

Casus belli types need the following localisation keys:

- `<key>`
- `<key>_desc`

Icon

In order to have an icon assigned to the casus belli, one need to create a new `.dds` entry entitled with the casus belli key in the directory based on `CASUS_BELLI_ICON_PATH` define (Vanilla value: `"gfx/interface/icons/casus_belli"`). If no icon is provided, the game will load under `ICON_DEFAULT_NAME` define entry (Vanilla value: `"_default"`).

Wargoal types

Wargoal types are assigned to casus bellis and can be shared between themselves. The main purpose of wargoal types is to define the war name formatting, rules for peace treaties, how and how much of warscore is gained.

Technical details

Wargoal types are located in `common/wargoals`, usually in the `in_game` top folder.

For example:





Syntax example

```
religious_conquer_province = {
    type = take_province

    attacker = {
        conquer_cost = 1.2
        subjugate_cost = 1.2
    }
    defender = {
    }
    ticking_war_score = 0.5
}
```

Ticking warscore

The behavior of the ticking war score can be modified using type. The following table represents the valid options for behavior and describes what each one represents:

Wargoal type	Behavior
take_province	Provides ticking warscore if the attacker country controls the entire target province.
take_capital	Provides ticking warscore if the attacker country controls the entire target province.
take_border	Provides ticking warscore if the attacker country controls the entire target province.
take_country	Provides ticking warscore if the attacker country controls the entire target country.
naval_superiority	The ticking warscore is decided by the side that has is able to blockade the enemy. If both sides are blockading, noone is considered to have the advantage. Moreover, won battles on sea give additional warscore according to the <u>DEFAULT_WARGOAL_BATTLESCORE_BONUS</u> define (Vanilla value: 3) define.
superiority	The ticking warscore is decided by the side that has more than <u>SUPERIORITY_WARGOAL_WARSCORE_THRESHOLD</u> define (Vanilla value: 10) % warscore in battles won. Moreover, won battles on land give additional warscore according to the <u>DEFAULT_WARGOAL_BATTLESCORE_BONUS</u> define (Vanilla value: 3) define.
enforce_military_access	The behavior is the same as take_province, with an extra text field.
defend_capital	Provides ticking warscore if the attacker country controls the entire target province.
independence	Provides ticking warscore if the attacker country controls the entire target province.
destroy_army	Ticks warscore when the opposite's army size is equal to 0.

ticking_war_score can also be used to set the value the warscore may tick to. The default is 1.0.

War name





- `$NUM$` representing the order of the war declared with this title e.g "First", "Second"
- `$ORDER$` represents the suffix of the `$NUM$`. Should always be combined with it: `$NUM$$ORDER$`
- `$FIRST$` representing the adjective name of the attacking country.
- `$FIRSTNAME$` representing the name of the attacking country.
- `$SECOND$` representing the adjective name of the defending country.
- `$SECONDNAME$` representing the name of the defending country.

Additionally, `war_name_is_country_order_agnostic = yes` can be set in order for the game to recognize the numbering independent of who declared the war, e.g "Second Phase of the Hundred Years War" will not depend on which side declared the first or second war.

If no name is provided, the game will default to `NORMAL_WAR_NAME`.

Peace parameters for attacker and defender

`attacker` and `defender` fields are used to set peace treaty costs and behavior for both sides of the conflict.

Syntax

```
example_wargoal = {
  ...
  attacker = {
    conquer_cost = 0.75
    subjugate_cost = 0.75
    allowed_locations = {
      custom_tooltip = {
        text = casus_belli_forbids_non_french_location_conquest_tt
        OR = {
          scope:location.region = region:france_region
          scope:location = {
            owner ?= { is_subject_of = c:FRA }
          }
        }
      }
    }
  }
  ...
}
```

Costs

Warscore costs for conquest of new locations and subjugation of countries is set using `conquer_cost` and `subjugate_cost`. Both are floating point numbers, so they cannot be made dynamic.

Moreover, antagonism costs for the peace offers can be changed using `antagonism`, also accepting floating point numbers. The antagonism cost reduction is taken account in cession of locations, areas, provinces, subjugation and union formation.

wed locations and subjugation



representing the location.

`allowed_subjugation` is used whether the country can be subjugated in a war. The `ROOT` scope in the trigger is the subjugated country, with `scope:winner` provided as the subjugating country. `scope:loser` is also provided, but represents the same country as `ROOT`.

Rules for subject/overlord

Most wars will automatically make the attacked overlord call in all of their subjects. This behavior can be disabled by using `call_in_subjects = no`.

Analogically, most wars will automatically make the attacked subject call in their overlord. This behavior can also be overridden to be disabled using `call_in_overlord = no`.

Required wargoal types

Some wargoal types are used internally in the game but are open for scripting. Those wargoal types should not be removed, otherwise the game will error and might behave unexpectedly:

- `take_country`
- `conquer_province`
- `take_province_tribal_feud`
- `take_country_nationalist`
- `take_capital`
- `take_capital_force_migration`
- `take_capital_imperial`
- `take_border`
- `independence`
- `superiority`
- `superiority_raiding`
- `superiority_heretic`
- `superiority_push_back_colonizers`
- `superiority_horde`
- `humiliate`
- `naval`
- `demand_military_access`

Localisation

Wargoal types need the following localisation keys:

- `war_goal_<key>`
- `war_goal_<key>_desc`





load under ICON_DEFAULT_NAME define entry (Vanilla value: "_default").

Scripted peace treaties

Europa Universalis V allows for creation of new, scripted peace treaties.

Technical details

Scripted peace treaties are located in `common/peace_treaties`, usually in the `in_game` top folder.

For example:

```
common/peace_treaties/example_file.txt
```

Syntax

```
disband_kontor = {
  cost = {
    add = {
      desc = "DIPLOREASON_BASE"
      value = "scope:target.location_peace_cost(scope:loser|scope:winner)"
      multiply = 0.75
    }
  }
  base_agonism = 1
  antagonism_type = antagonism_disband_kontor
  potential = {
    scope:war = {
      or = {
        has_casus_belli = no
        casus_belli = {
          is_trade_cb = no
        }
      }
    }
  }
  select_trigger = {
    looking_for_a = location
    source = recipient
    target_flag = target
    visible = {
      location_building_level = {
        building_type = building_type:hanseatic_kontor
        value > 0
        owner = scope:recipient
      }
    }
  }
  effect = {
    scope:target = {
      destroy_building = "building(building_type:hanseatic_kontor|scope:recipient)"
    }
  }
  ai_desire = {
    value = 10
  }
}
```

Requirements and effects



scope:loser and scope:war scopes.

Effect is executed using effect. The scopes here use the full set of targets from scopes available section.

Peace treaty category

Peace treaty category can be set using category. If not set, this can also be set implicitly based on what scope the select_trigger is looking for.

The category is used to compartmentalize the peace treaty into one of the peace treaty categories.

The following categories exist:

- country
- location
- province
- area
- dismantlefort
- cores
- releasecountry
- cancelsubject
- seizeland
- misc - the default.

AI

The likelihood of AI using the scripted peace treaty is decided using ai_desire script value - this scripted value will use the full set of targets from scopes available section.

Peace treaty warscore and antagonism cost

Peace treaty cost in warscore is decided by the cost scripted value - this scripted value will use the full set of targets from scopes available section.

Peace treaty antagonism cost is decided via base_antagonism scripted value field, this scripted value uses the same targets as cost. Moreover, this field must be coupled with antagonism_type which is a key of a bias type, which determines the category of antagonism this peace treaty causes - as well as its decay and so forth.

Interaction targets in peace treaties

Scripted peace treaties utilize the Interaction target syntax, but in a different matter - they are supposed to generate multiple versions of the same peace deal. A peace deal to dismantle fort use an interaction target looking for locations with a fort.

There can be only one interaction target in peace treaty types.



- location
- province
- province_definition
- area
- international_organization

Winning country is represented as `scope:actor` and the losing country is `scope:recipient`.

Scopes available in scripted peace treaties

Most fields except for potential and allow allow for full use of `scope:winner`, `scope:loser`, `scope:war` and the interaction target, if present:

- `scope:winner` - country representing the winning country
- `scope:loser` - country representing the losing country
- `scope:war` - war representing the war
- Interaction target scope based on the interaction target, if present

Additional fields

There are two additional fields that can be used in peace treaties:

- `blocks_full_annexation` = `yes`, which blocks the target from being fully annexed when this peace treaty is selected.
- `are_targets_exclusive` = `yes` is used to make location and province target treaties exclusive with cession treaties. For example, do not allow to both take land and dismantle forts in it.

Localisation

Scripted peace treaties need the following localisation keys:

- `<key>_entry` - longer entry name, shown mainly in tooltip of the entry
- `<key>_desc` - the text when calling `.GetDesc`
- `<key>` - the text when calling `.GetName`
- `<key>_entry_short` - the text shown in peace treaty view

The country taking is provided with `COUNTRY`. It can be referred to as such: `[COUNTRY.GetName]`. Similarly, the losing country is provided with `TARGET_COUNTRY`. Moreover, peace treaties utilizing interaction targets will have the target flag provided for use.

Icon

In order to have an icon assigned to the peace treaty, one need to create a new `.dds` entry named with the peace treaty key in the directory based on `TREATY_TYPE_ICON_PATH` name (Vanilla value: `"gfx/interface/icons/treaty_type/"`). If no icon is provided, the



Modding

[Return to top](#)**Documentation**

[Defines](#) • [Effects](#) • [Scopes](#) • [Scope links](#) • [Triggers](#) | [Colors](#) • [Macros](#) • [Mean time to happen](#) • [Modifier types](#) • [On actions](#) • [Script value](#) • [Variables](#) | [GUI script](#) • [Localization](#)

Scripted content

[Actions](#) • [Disasters](#) • [Events](#) • [Missions](#) • [Modifiers](#) • [Scripted gui](#) • [Setup](#) • [Situations](#) • [Customizable localization](#)

Scripted types

[Advances](#) • [Art](#) • [Buildings](#) • [Bureaucracies](#) • [Casus belli](#) • [Characters](#) • [Concepts](#) • [Countries](#) • [Culture](#) • [Diplomacy](#) • [Diseases](#) • [Estates](#) • [Goods](#) • [Institutions](#) • [International organizations](#) • [Laws](#) • [Movements](#) • [Peace treaties](#) • [Pops](#) • [Religion](#) • [Subject types](#) • [Traits](#) • [Units](#) • [Wargoals](#)

Map

[Map](#) • [Map modes](#) • [Terrain](#)

Graphics

[3D Models](#) • [Interface](#) • [Graphical assets](#) • [Fonts](#) • [Flags](#)

Audio

[Music](#) • [Sound](#)

Other

[AI](#) • [Console commands](#) • [Checksum](#) • [Mods](#) • [Mod compatibility](#) • [Mod structure](#) • [Troubleshooting](#)

Guides

[Interface modding guide](#) • [Mod translation](#) • [Save-game editing](#) • [Settlement position modding guide](#)

Tools

[Arcanum](#) • [PDX DeepL](#) • [PDX Workshop Manager](#) • [Community Mod Toolkit](#) • [Add Your Tool to the Wiki](#)

"https://eu5.paradoxwikis.com/index.php?title=War_modding&oldid=33014" sayfasından alınmıştır

Sayfa en son 22.41, 10 Mart 2026 tarihinde değiştirildi.

Aksi belirtilmedikçe içeriğin kullanımı Attribution-ShareAlike 3.0 lisansı kapsamında uygundur.



GAMES

[Discover](#)

[Our Brands](#)

[Browse](#)

[Play on Paradox technology](#)

ABOUT

[News](#)

[About us](#)

[Careers](#)

[Join our playtests](#)

[Media contact](#)

COMMUNITY

[Paradox Forums](#)

[Paradox Wikis](#)

[Support](#)

SOCIAL MEDIA

[Terms & Policies](#)

[Legal Information](#)

[EU Online Dispute Resolution](#)

[Report Illegal Content](#)

[Frequently Asked Questions](#)

[Paradox Interactive corporate website](#)

